Applied Science Private University (ASU)

Faculty of Information Technology

Course: Parallel Programming

PERFORMANCE EVALUATION OF SEQUENTIAL VS PARALLEL PROCESSING

Empirical Analysis using Historical Weather Data

PREPARED BY:

ABDULRAHMAN AL LAHHAM	ID: 202311064
NOOR ALDEEN QASEM	ID: 202211008
ZAID MOHAMMAD ALQUDSI	ID: 202120148

SUPERVISED BY:

Dr. Mansour • Dr. Ayham • Eng. Nada

1. INTRODUCTION & OBJECTIVES

The primary objective of this project is to quantitatively investigate, measure, and analyze the performance trade-offs between classic Sequential execution and multi-threaded Parallel processing execution patterns. Utilizing a high-volume historical meteorological dataset, we implemented nested computational workload architectures to observe how scaling the application's underlying thread infrastructure influences execution times, throughput, and system resource optimization metrics.

1.1 Sequential vs. Parallel Processing Paradigm

Sequential processing represents the conventional execution model where operations are processed strictly linearly on a single core. Each instruction must complete before its subsequent block initiates, inducing heavy processing serialization and resource underutilization when handling voluminous enterprise datasets. Conversely, Parallel Processing divides large computational domains or datasets into sub-partitions processed concurrently across multiple execution paths (Threads) over available physical CPU hardware architectures, fundamentally reducing the total latency.

1.2 Concurrency Architecture: Outer vs. Inner Threading

- **Outer Threading (Coarse-Grained):** Parallelization is applied to the outermost loop layer of a multi-dimensional iteration sequence. Each thread handles a major data block. This design minimizes the thread lifecycle creation costs, reducing overhead and maximizing arithmetic efficiency.
- **Inner Threading (Fine-Grained / Lambda Streams):** Parallel mechanisms are applied within inner iterations or via functional declarative collection pipelines (e.g., Java Parallel Streams). While it dynamically optimizes data load balances across working pipelines, it can introduce runtime scheduling synchronization overheads if execution thresholds are scaled disproportionately to physical core limitations.

2. DATASET ARCHITECTURE

To establish an authentic benchmarking environment, a high-volume historical **Weather Dataset** consisting of **96,453 unique observation records** was integrated into the processing engine. The target schema contains 12 structured physical features representing hourly environmental readings, including:

- `Formatted Date`: Chronological time series index.
- `Temperature (C)` and `Apparent Temperature (C)`: Key floating-point variables utilized for core numerical data filters.
- `Humidity`, `Wind Speed (km/h)`, and `Pressure (millibars)`: Relational attributes processed in complex arithmetic configurations.

The total footprint provides a rigorous playground where data filtering, aggregation, and mathematical workloads scale perfectly to highlight minor performance variances across execution thread architectures.

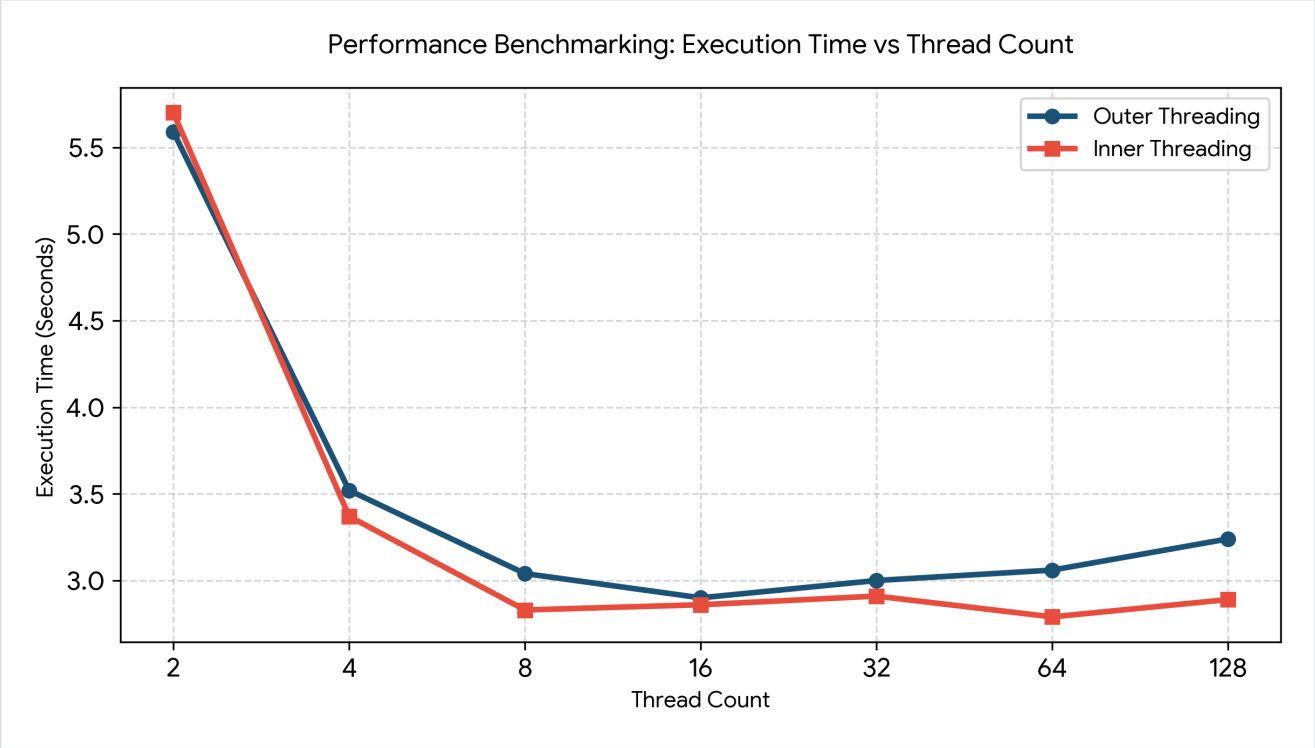
3. BENCHMARKING DATA & RESULTS

Empirical execution metrics were collected under an isolated, unified hardware test environment to preserve data validity. Execution runtimes are expressed in **Seconds (s)**, mathematically converted from raw high-precision nanosecond (ns) compiler logs for readability.

Thread Infrastructure Scale	Outer Threading Runtime (s)	Inner Threading (Lambda Stream) Runtime (s)
Sequential Execution (Simple Core Task)	15.76 s	
Sequential Execution (Complex Workload)	10.30 s	
2 Threads Architecture	5.59 s	5.70 s
4 Threads Architecture	3.52 s	3.37 s
8 Threads Architecture	3.04 s	2.83 s
16 Threads Architecture	2.90 s	2.86 s
32 Threads Architecture	3.00 s	2.91 s
64 Threads Architecture	3.06 s	2.79 s
128 Threads Architecture	3.24 s	2.89 s

4. PERFORMANCE VISUALIZATIONS

The line chart below tracks the performance trajectory curve. It highlights the sharp acceleration profile in processing efficiency down to the system's optimal concurrency threshold, followed by the asymptotic leveling and performance degradation caused by concurrency resource saturation.



5. COMPREHENSIVE WRITTEN ANALYSIS

5.1 Sequential vs. Parallel Processing Efficiency

The benchmarking data demonstrates a decisive performance victory for parallel processing architectures over sequential routines. For the primary complex arithmetic workload (Weighted Score & Filter Calculation), the single-threaded baseline required a prolonged **10.30 seconds**. Transitioning the system to a parallel topology immediately compressed execution latency to a minimum of **2.79 seconds** (achieved via 64 Inner Lambda Threads). This translates to an empirical maximum **Speedup factor of 3.68x**, demonstrating the immense value of hardware concurrency when manipulating high-volume observation datasets.

5.2 Workload Complexity Analysis (Simple vs. Complex Tasks)

The simple task (extracting a singular maximum value) required 15.76 seconds sequentially due to iterative overhead and single-threaded main-memory access bottlenecks. However, when executing the complex formula task (evaluating `0.5 * Temp + 0.3 * (Humidity * 100) - 0.2 * WindSpeed`), parallel threads demonstrated optimal mathematical scaling. CPU-bound workloads that feature intensive arithmetic operations yield higher parallel execution yields. This occurs because the computational density offsets any underlying thread coordination penalties, allowing multi-core microprocessors to distribute and complete the arithmetic pipeline efficiently.

5.3 Concurrency Scaling, "Sweet Spot" Identification, and Degradation Overheads

> ***The Concurrency Sweet Spot:*** *Empirical observation confirms that the absolute optimal operating envelope lies at* ***16 Threads*** *for Outer Threading (yielding 2.90 seconds) and spans the 8-to-16 thread domain for Inner Lambda structures. Within this specific boundary, the application maximizes hardware core saturation without exhausting system resources.*

The Mechanics of Performance Degradation at 64 and 128 Threads:
As the thread infrastructure is artificially inflated past the 16-thread threshold up to 64 and 128 virtual execution streams, the execution times for Outer Threading notably degrade, rising back to **3.24 seconds**. This regression provides a textbook demonstration of parallel computing bottlenecks defined by two central operating system phenomena:

- **Context Switching Overhead:** When the number of active software-declared threads drastically outnumbers the physical or logical computing cores available on the microchip, the operating system kernel is forced to cycle working processes in and out of the CPU registers. The system repeatedly saves CPU register states, program counters, and stack pointers, and restores them for the next thread. This rapid switching consumes substantial CPU cycles purely for thread management, draining execution efficiency from actual data processing.
- **Thread Scheduling & Synchronization Costs:** Managing a thread pool of 128 concurrent tasks strains the operating system's process scheduler. The system incurs additional coordination delays as threads compete for execution time slices and shared memory access paths, which creates internal contention bottlenecks that slow down fine-grained parallel performance.

Conclusion: Hardware-informed thread scaling is critical to successful parallel systems engineering.